BRAINERX · AI MASTERCLASS

Decision Trees

What Is a DT · Anatomy · Gini & Entropy

Manual Construction · Hyperparameters · Overfitting

Titanic Example · Visualization · Pros & Cons

What We'll Cover Today

Every aspect of Decision Trees — explained from scratch with real examples

01 **The Big Idea**
What is a DT? Why do we use it? Real-world analogy

02 **Anatomy & Terminology**
Root, internal nodes, leaves, branches, depth — all defined

03 **Supervised Learning**
How DTs learn from labeled data. What 'training' means

04 **The Splitting Engine**
Gini Impurity — the math behind every split decision

05 **Entropy & Info Gain**
ID3 algorithm. Entropy formula. Information Gain ranking

06 **Manual Construction**
Build a tree by hand — step by step with 7 bank clients

07 **Titanic Application**
Apply a DT to real Titanic survival data. Read the tree.

08 **Hyperparameters**
max_depth, min_samples_leaf, criterion, random_state

09 **Overfitting & Pruning**
Why DTs overfit, how to stop it, visual demonstration

10 **Strengths & Weaknesses**
When to use a DT and when NOT to use it

11 **Visualization & Code**
plot_tree(), sklearn code, how to read every node

12 **Summary & Cheatsheet**
All formulas, rules, and tips on one slide

01

The Big Idea

What is a Decision Tree and why should we care?

Decision Trees Are Everywhere in Real Life

You already use decision trees every day — without knowing it

Should I bring an umbrella today?

YES

Is it raining?

YES

☂ Take umbrella!
(it's raining now)

NO

☂ Take umbrella!
(better safe)

NO

Will it rain later?

NO

☀ No umbrella!
(enjoy the sun)

YES

Check the app
(uncertain)

This is EXACTLY what a Machine Learning Decision Tree does — but with data instead of personal knowledge.

Instead of 'Is it raining?' it asks 'Is passenger female?' 'Is Fare > 50?' 'Is Pclass = 1?'

The tree LEARNS these questions automatically from thousands of examples.

What Is a Decision Tree — Formal Definition

A supervised ML algorithm that makes predictions through a series of yes/no questions

A Decision Tree is a supervised learning algorithm that recursively partitions data using the most informative feature at each step, creating a hierarchical set of IF-THEN rules that lead to a prediction.

Supervised Learning

Learns from LABELED training data — rows where we know the answer. Uses those examples to build rules.

Fully Interpretable

You can DRAW the tree and follow every decision path. No black box — total transparency.

Predicts CATEGORIES

Survived vs. Not Survived · Spam vs. Not Spam · Disease vs. Healthy

No Feature Scaling

Trees use THRESHOLDS (Age > 15, Fare > 50). Not distances. Scaling doesn't matter.

Predicts NUMBERS

House price · Temperature tomorrow · Number of days until default

Non-Parametric

No assumptions about the data distribution. Works with skewed, non-normal, mixed data.

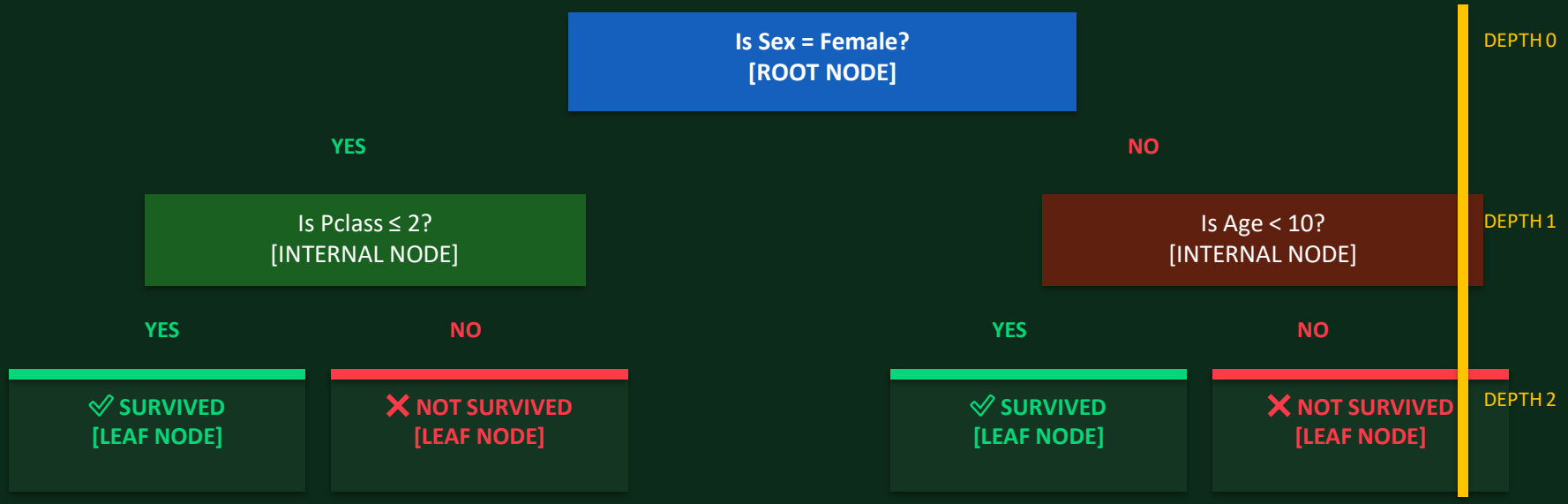
02

Anatomy & Terminology

Every part of a Decision Tree — named and explained

Anatomy of a Decision Tree

Learn these 6 terms — they appear everywhere in ML literature



Root Node

The TOP node. The single starting point. Only 1 per tree. Contains the BEST first question.

Internal Node

A decision point inside the tree. Asks a question, then splits into 2 branches.

Leaf Node

The END point. No more splitting. This is the PREDICTION output.

Branch

An arrow connecting parent to child. Represents one answer (YES or NO) to the question.

Depth

Number of levels from root to a leaf. Controls tree complexity.

Decision Trees as Supervised Learning

Understanding where DTs fit in the ML landscape

Supervised Learning = learning FROM EXAMPLES where we already know the right answer

TRAINING PHASE

We show the algorithm 668 passengers
For each passenger: features + label
Algorithm asks: which question best separates survivors from non-survivors?
→ Builds the tree structure

↔

PREDICTION PHASE

New passenger arrives (from test set)
Their features: female, 1st class, age 28
Tree asks questions one by one
Follows YES/NO branches down
→ Reaches leaf: SURVIVED

Features (X) = Inputs

Sex, Age, Fare, Pclass, SibSp, Parch, Embarked

Target (y) = Output

Survived: 0 (did not survive) or 1 (survived)

03

The Splitting Engine

Gini Impurity — the math that drives every split

Impurity — The Key Concept Behind Every Split

Before we learn the formula, we need to understand WHAT we are measuring

Impurity measures HOW MIXED a group is. A pure group = all same class. An impure group = mixed classes.

PURE GROUP

10 balls, all RED

Impurity = 0.0
(Perfect!)

Gini = 0.0

MIXED GROUP

5 red, 5 blue balls

Impurity = 0.5
(Worst case!)

Gini = 0.5

PARTIAL GROUP

8 red, 2 blue balls

Impurity = 0.32
(Mostly pure)

Gini = 0.32

The CART algorithm tries EVERY possible split and picks the one that creates the PUREST groups.

This is done by minimizing the WEIGHTED average impurity of the two child groups.

Gini Impurity Formula — Step by Step

Breaking down the formula into simple pieces

$$\text{Gini(node)} = 1 - \Sigma(p_i)^2$$

Where p_i = proportion (probability) of class i in the node

Step 1
Count classes

Count how many of each class exist in the node.
Example: 40 survived (class 1), 10 not survived (class 0)
Total = 50 passengers in this node

Step 2
Compute proportions

p(survived) = 40/50 = 0.80
p(not survived) = 10/50 = 0.20
Each proportion tells us the % of that class

Step 3
Square each

$p(\text{survived})^2 = 0.80^2 = 0.64$
$p(\text{not survived})^2 = 0.20^2 = 0.04$
(Squaring makes larger proportions contribute more)

Step 4
Subtract from 1

Gini = 1 – (0.64 + 0.04)
Gini = 1 – 0.68
Gini = 0.32
→ Mostly pure, but not perfect

The Cost Function — Choosing the Best Split

How CART compares all possible splits to find the winner

```
Cost = (N_left / N_total) × Gini_left + (N_right / N_total) × Gini_right
```

This is a WEIGHTED average — bigger groups have more influence on the cost

Example: Split on 'Sex = Female?'

```
Total: 668 training passengers

Left branch (Female = YES): 234 passengers
  → 193 survived, 41 not survived
  → p(survived) = 193/234 = 0.824
  → Gini_left = 1 – (0.824² + 0.176²) = 0.290

Right branch (Female = NO): 434 passengers
  → 109 survived, 325 not survived
  → p(survived) = 109/434 = 0.251
  → Gini_right = 1 – (0.251² + 0.749²) = 0.376
```

Cost Calculation:

```
Cost = (234/668) × 0.290
     + (434/668) × 0.376

Cost = 0.350 × 0.290
     + 0.650 × 0.376

Cost = 0.101 + 0.244

Cost = 0.346
```

CART: Classification And Regression Tree. It's the algorithm sklearn uses by default.

→ We test ALL features this way

→ Pick the LOWEST Cost!

Entropy & Information Gain — The ID3 Alternative

A different but equally valid way to measure impurity

Entropy Formula (ID3):

$$E(S) = -\Sigma\, p_i \times \log_2(p_i)$$

Information Gain:

$$Gain(T,X) = E(T) - E(T,X)$$

PURE: Entropy = 0

10 Yes, 0 No

$E = -(1\times\log_2 1) - (0\times\log_2 0)$
$E = 0.0$ ✓ No uncertainty

50/50: Entropy = 1

5 Yes, 5 No

$E = -(0.5\times\log_2 0.5) \times 2$
$E = 1.0$ ✗ Max uncertainty

80/20: Entropy = 0.72

8 Yes, 2 No

$E = -0.8\times\log_2 0.8 - 0.2\times\log_2 0.2$
$E = 0.72$ ◈ Moderate

Property	Gini Impurity	Entropy	Verdict
Used by	CART algorithm (sklearn default)	ID3, C4.5, C5.0	Both widely used
Range	0.0 → 0.5	0.0 → 1.0	Entropy has bigger range
Speed	Faster (no logarithm)	Slower ($\log_2$ calculation)	Gini is faster
Result	Very similar splits	Very similar splits	Rarely matters in practice

04

Manual Construction

Building a Decision Tree by hand — step by step

The Dataset — Bank Loan Decision Problem

7 clients. We want to learn: who should get a loan? Let's build the tree by hand.

ID	Debt	Job	Salary (k€)	Decision
1	Low	Stable	85	✅ Accept
2	High	Stable	90	❌ Refuse
3	Low	Unstable	45	❌ Refuse
4	Low	Stable	120	✅ Accept
5	High	Unstable	60	❌ Refuse
6	Low	Stable	110	✅ Accept
7	Low	Stable	50	❌ Refuse

Goal: Find the question (feature + threshold) that reduces Gini the most

Step 1: Measure Root Gini

```
All 7 clients together:
  • 3 Accepted (class 1): #1, #4, #6
  • 4 Refused  (class 0): #2, #3, #5, #7

p(Accepted) = 3/7 = 0.4286
p(Refused)  = 4/7 = 0.5714

Gini_root =
  1 – (0.4286² + 0.5714²)
= 1 – (0.1837 + 0.3265)
= 1 – 0.5102
≈ 0.490

→ Very impure! (close to 0.5)
→ Must split to reduce impurity
```

Round 1 — Testing Split: Debt = High vs Low?

Testing Feature #1: Does splitting on Debt give us a good split?

Split on DEBT

```
LEFT branch: Debt = High
  Clients: #2 (Refuse), #5 (Refuse)
  → 0 Accepted, 2 Refused
  → p(Refused) = 1.0, p(Accepted) = 0.0
  → Gini_left = 1−(1²+0²) = 0.0  ✓ PURE!

RIGHT branch: Debt = Low
  Clients: #1(A), #3(R), #4(A), #6(A), #7(R)
  → 3 Accepted, 2 Refused
  → p(A) = 3/5 = 0.6, p(R) = 2/5 = 0.4
  → Gini_right = 1−(0.6²+0.4²) = 0.48

Weighted Cost (Debt):
  = (2/7)×0.0 + (5/7)×0.48
  = 0 + 0.343
  = 0.343
```

Split on JOB

```
LEFT branch: Job = Unstable
  Clients: #3 (Refuse), #5 (Refuse)
  → 0 Accepted, 2 Refused
  → Gini_left = 1−(1²+0²) = 0.0  ✓ PURE!

RIGHT branch: Job = Stable
  Clients: #1(A), #2(R), #4(A), #6(A), #7(R)
  → 3 Accepted, 2 Refused
  → p(A) = 3/5, p(R) = 2/5
  → Gini_right = 0.48

Weighted Cost (Job):
  = (2/7)×0.0 + (5/7)×0.48
  = 0.343

Tie! Debt and Job cost the same.
We test Salary next to compare...
```

Round 1 — Testing Split: Salary < 70k? (The Winner!)

Salary is a numerical feature — we test threshold splits

Split on SALARY < 70k

```
Sort salary values: 45, 50, 60, 85, 90, 110, 120
Test threshold: < 70k

LEFT branch: Salary < 70k
  Clients: #3(45k,R), #5(60k,R), #7(50k,R)
  → 0 Accepted, 3 Refused
  → p(R) = 1.0, p(A) = 0.0
  → Gini_left = 1-(1²+0²) = 0.0  ✓✓ PURE!

RIGHT branch: Salary ≥ 70k
  Clients: #1(85k,A), #2(90k,R), #4(120k,A), #6(110k,A)
  → 3 Accepted, 1 Refused
  → p(A) = 3/4 = 0.75, p(R) = 1/4 = 0.25
  → Gini_right = 1-(0.75²+0.25²) = 0.375

Weighted Cost (Salary < 70k):
  = (3/7)×0.0 + (4/7)×0.375
  = 0 + 0.214
  = 0.214  ← LOWEST!
```

Competition Results:

Debt = High/Low

Cost = 0.343

Rank 2

Job = Stable/Unstable

Cost = 0.343

Rank 2

Salary < 70k

Cost = 0.214

🏆 **WINNER!**

Salary < 70k wins Round 1 → becomes the ROOT NODE

Round 2 — Splitting the Right Branch Further

The left branch (Salary < 70k) is pure. Now focus on the right branch: clients 1, 2, 4, 6

Right Branch: Salary ≥ 70k

```
Client #1: Debt=Low, Job=Stable, Sal=85
→ ✅ Accept
Client #2: Debt=High, Job=Stable,
Sal=90  → ❌ Refuse
Client #4: Debt=Low, Job=Stable,
Sal=120  → ✅ Accept
Client #6: Debt=Low, Job=Stable,
Sal=110  → ✅ Accept

Gini of this group:
  p(A)=3/4=0.75, p(R)=1/4=0.25
  Gini = 1 - (0.75²+0.25²) = 0.375
```

Which feature splits these 4 clients best?

```
Job? → All 4 are Stable → Zero variance → No split
possible

Salary?
  → < 97k: #1(85), #4(not in range)... complex, won't help

Debt? → The KEY question!
  Left (Debt=High): Client #2 only → 0 Accept, 1 Refuse
    Gini_left = 0.0  ✅ PURE!
  Right (Debt=Low): #1, #4, #6 → 3 Accept, 0 Refuse
    Gini_right = 0.0  ✅ PURE!
```

The Final Tree — 3 Simple Rules

```
Rule 1: IF Salary < 70k  THEN → REFUSE  (clients 3, 5, 7 → all correctly refused)

Rule 2: IF Salary ≥ 70k AND Debt = High  THEN → REFUSE  (client 2 → correctly refused)

Rule 3: IF Salary ≥ 70k AND Debt = Low   THEN → ACCEPT  (clients 1, 4, 6 → all correctly accepted)
```

The Final Decision Tree — Bank Loan Approval

A perfectly pure tree built from 7 clients — zero training errors

Salary < 70k €?

YES

✗ **REFUSED**
clients 3, 5, 7

NO

Debt = High?

YES

✗ **REFUSED**
client 2

NO

✓ **ACCEPTED**
clients 1, 4, 6

What the Model Discovered Automatically:

- SALARY is the single most important factor — determines 3 out of 7 decisions alone
- DEBT is used to resolve ambiguity among high-salary clients
- JOB was completely IGNORED — it adds zero value once Salary and Debt are known
- The algorithm proved: low salary = always refused | high salary + low debt = always accepted

05

Titanic Application

Applying a Decision Tree to real survival data

The Titanic Dataset — Our ML Problem

Binary classification: predict whether a passenger survived based on 8 features

Feature	Type	Values	Why Useful?
Pclass	Ordinal	1, 2, or 3	1st class = more lifeboats
Sex	Binary	male / female	'Women first' policy
Age	Numerical	0 to 80 years	'Children first' policy
SibSp	Numerical	0 to 8	Family size affects escape
Parch	Numerical	0 to 9	Parents/children aboard
Fare	Numerical	£0 to £512	Higher fare = wealthier
Embarked	Categorical	C, Q, or S	Port → affects cabin location

Target Variable: Survived

0 = Did not survive (549 passengers, 62%) 1 = Survived (342 passengers, 38%)

Historical Context:

April 15, 1912 — RMS Titanic sank after hitting an iceberg.

Key historical facts that SHOULD show up in our tree:

1. 'Women and children first'
   → Sex should be ROOT

2. 1st class passengers had better access to lifeboats
   → Pclass should appear

3. Higher fare = 1st class
   → Fare and Pclass linked

4. Children prioritized
   → Age < 10 should appear

The Decision Tree Learned from Titanic Data

sklearn's output — max_depth=4. Reading every node.

Sex ≤ 0.5
(Is passenger female?)

YES (female)

Pclass ≤ 2.5
(Is Pclass 1 or 2?)

YES

✓ **SURVIVED**
F1: 0.93

NO

✓ **SURVIVED**
F1: 0.77

NO (male)

Age ≤ 9.5
(Is child < 10?)

YES

✓ **SURVIVED**
children!

NO

✗ **DID NOT SURVIVE**

How to Read a Node in sklearn's plot_tree():

`Sex ≤ 0.5`	The QUESTION. (Sex: female=0, male=1). If Sex ≤ 0.5 → passenger is female. LEFT branch = YES.
`gini = 0.32`	Current IMPURITY of this node (lower = purer). Root node Gini ≈ 0.49.
`samples = 668`	How many training passengers reached this node.
`value = [411, 257]`	Count of [class 0 (not survived), class 1 (survived)] in this node.
`class = 0`	The MAJORITY CLASS. If we stopped here, we'd predict 'Did not survive'.

Titanic Decision Tree — Training & Evaluation

What accuracy did we get? How do we measure it correctly?

Classification Report (Decision Tree, max_depth=4)

	precision	recall	f1-score	support
Did Not Survive	0.8490	0.8657	0.8573	134
Survived	0.7912	0.7640	0.7774	89
accuracy			0.8251	223
macro avg	0.8201	0.8149	0.8173	223
weighted avg	0.8244	0.8251	0.8246	223

Confusion Matrix:

	Pred: No	Pred: Yes
Act: No	116 TN	18 FP
Act: Yes	21 FN	68 TP

Reading These Numbers:

TN=116 → Correctly predicted 116 'Did not survive'

TP=68 → Correctly predicted 68 survivors

FP=18 → Wrongly said 'Survived' — actually didn't

FN=21 → Missed 21 actual survivors (worst!)

Accuracy = (116+68)/223 = 82.5%

Precision = 68/(68+18) = 79.1%

06

Hyperparameters

The knobs you control to shape how the tree grows

max_depth — The Most Important Hyperparameter

Controls how many questions the tree can ask before making a prediction

max_depth = the maximum number of levels from root to any leaf. Depth 0 = just the root (1 node).

depth=1

Underfit

1 question

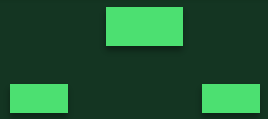
Only asks ONE question.
Very underfit — misses most patterns.
Train ~65%, Test ~63%

depth=4

Just Right ✓

4 questions

4 levels of questions.
Good balance of complexity.
Train ~90%, Test ~83%



depth=None

Overfit ⚠

Unlimited

Grows until every leaf is pure.
Memorizes every training example.
Train ~100%, Test ~72%

✓ Recommended: max_depth = 3 to 6 for most real datasets. Start with 4, tune with cross-validation.

All Key Hyperparameters — Complete Guide

Understanding what each setting controls and how to choose it

Parameter	Type	Default	Best	What it does
`max_depth`	int or None	4	3–6	Max levels. None = unlimited (overfits). MOST important parameter.
`min_samples_leaf`	int	1	10–20	Each leaf must have ≥N samples. Prevents tiny leaves that memorize 1-2 outliers.
`min_samples_split`	int	2	10–20	Min samples needed to split a node. Prevents splitting groups that are already small.
`criterion`	string	`'gini'`	`'gini'`	Impurity measure. 'gini' (CART) is faster. 'entropy' (ID3) is slightly different. Results are similar.
`max_features`	int/None	None	`None` or `'sqrt'`	# features to consider at each split. None = all features. 'sqrt' adds randomness.
`random_state`	int	None	42	Random seed for reproducibility. ALWAYS set this during development!
`class_weight`	dict/None	None	`'balanced'`	Compensates for class imbalance. 'balanced' auto-adjusts based on class frequencies.
`max_leaf_nodes`	int/None	None	50–200	Hard limit on total leaf nodes. Alternative way to control complexity.

07

Overfitting & Pruning

The #1 problem with Decision Trees — and how to fix it

BrainerX · Decision Trees — Complete Guide

Overfitting — When the Tree Memorizes Instead of Learning

The most common problem with Decision Trees — and why it matters

Overfitting = the model learns the TRAINING DATA perfectly but fails on NEW data it has never seen

Underfitting

`max_depth = 1`

Train accuracy: 62%
Test accuracy: 61%
→ Both low
→ Model too simple
Not asking enough questions to learn the real patterns

Just Right

`max_depth = 4`

Train accuracy: 90%
Test accuracy: 83%
→ Both high!
→ Gap is small (7%)
Good generalization
Captures real patterns without noise

Overfitting

`max_depth = None`

Train accuracy: 99%
Test accuracy: 72%
→ HUGE gap! (27%)
→ Model memorized every passenger
Gini=0 at ALL leaves
Useless on new data

Golden Rule: Always check BOTH train accuracy AND test accuracy. A big gap → overfitting.

Pruning — How to Control Overfitting

6 concrete techniques to prevent your tree from memorizing noise

✂ max_depth

Set a maximum depth limit

Most effective fix. Stops the tree from growing too deep.
max_depth=4 → tree can ask at most 4 questions.
Anything deeper probably captures noise, not signal.

🍃 min_samples_leaf

Minimum samples per leaf

Each leaf must represent at least N passengers.
min_samples_leaf=10 → no leaf with < 10 passengers.
Prevents learning rules that only apply to 1-2 people.

✂ min_samples_split

Minimum to split

A node must have at least N samples before it can split.
min_samples_split=20 → won't create tiny specialized branches.
Similar effect to min_samples_leaf but acts earlier.

🔢 max_leaf_nodes

Hard cap on leaves

Absolute limit on the number of leaf nodes in the entire tree.
max_leaf_nodes=30 → the tree has at most 30 predictions.
More intuitive than depth for some problems.

⚖ class_weight='balanced'

Handle imbalance

When one class (Not Survived 62%) is more common, the tree over-focuses on it. 'balanced' weights each class inversely proportional to its frequency.

🔄 Cross-Validation

Estimate true performance

Don't rely on a single train/test split. Use k-fold CV.
cross_val_score(dt, X, y, cv=5) → 5 different test sets.
Average score = true model performance.

08

Strengths & Weaknesses

Knowing when to use — and not use — a Decision Tree

Decision Trees — Complete Pros & Cons Analysis

Every algorithm has a right time and a wrong time. Know both.

✓ Strengths

- **Interpretable**

  You can draw and READ the tree. Explain every prediction. Legal/medical requirement.

- **No feature scaling**

  No need for StandardScaler. Works directly with raw features.

- **Handles mixed data**

  Works with numbers, categories, and text in the same dataset.

- **Fast prediction**

  Just follow branches — O(depth) = microseconds per prediction.

- **Feature importance**

  Tells you which features the model relied on most.

- **No assumptions**

  Doesn't assume normal distribution, linear relationships, etc.

- **Handles nonlinearity**

  Can learn complex nonlinear boundaries between classes.

- **Missing values**

  Some implementations handle NaN directly.

✗ Weaknesses

- **Overfitting**

  Unlimited trees memorize perfectly — always limit depth.

- **High variance**

  Small data change → completely different tree structure.

- **Biased to features**

  Features with more split points (numbers) get more attention.

- **Poor extrapolation**

  Can't predict values outside training range (regression).

- **Greedy splits**

  Local optimal decisions may not be globally optimal.

- **Unstable**

  One outlier can change the entire tree structure.

- **Rectangular boundaries**

  Can struggle with diagonal decision boundaries.

- **Replaced by RF/XGB**

  In practice, RF and XGBoost almost always outperform a single DT.

When to Use (and NOT Use) a Decision Tree

A practical guide for choosing the right algorithm

✓ USE a Decision Tree when:

• You need to EXPLAIN the model

A doctor, judge, or regulator needs to understand each prediction. Only Decision Trees (and Logistic Regression) provide this.

• Small to medium dataset

Under 10,000 rows? A single tree is fast, interpretable, and effective.

• Business rules

The output IS the rules: 'If age < 18 AND income < 30k → high risk.'

• Quick baseline model

Build a tree in 3 lines of code. Compare other models against it.

• Mixed feature types

Got numbers, categories, booleans all together? Trees handle them natively.

✗ DON'T use a Decision Tree when:

• High accuracy is critical

Random Forest or XGBoost will almost always beat a single tree by 3-8%.

• Very large datasets (>100k)

Training becomes slow. Random Forest (parallel) is faster.

• High-dimensional data

100+ features? Trees overfit badly. Use regularized models instead.

• Data is mostly continuous

Trees create stepwise boundaries. SVMs or neural nets may work better.

• Stability matters

Same data, slightly different split → different tree. Use Random Forest.

09

Visualization & Code

Writing and reading Decision Trees in Python

Complete Decision Tree Code — Titanic Example

Every line explained — from data prep to evaluation

Step 1: Imports

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
```

Step 2: Prepare X and y

```
# X = features (inputs), y = target (what we predict)
X = df[['Pclass','Sex','Age','SibSp','Parch','Fare','Embarked_Q','Embarked_S']]
y = df['Survived']  # 0 or 1
```

Step 3: Split train/test

```
# stratify=y keeps same class % in both splits
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

Step 4: Create & Train model

```
dt = DecisionTreeClassifier(
    max_depth=4,          # max levels (prevents overfitting)
    min_samples_leaf=10,  # each leaf needs ≥ 10 samples
    criterion='gini',     # impurity measure
    random_state=42       # reproducibility
)
```

How to Read plot_tree() Output

Decoding every number inside each node of the visualization

THE QUESTION being asked at this node.
Sex was encoded: female=0, male=1
Sex ≤ 0.5 means 'Is this passenger female?'
YES (≤0.5=female) → go LEFT | NO (>0.5=male) → go RIGHT

IMPURITY of this node BEFORE splitting.
Value of 0.47 is close to 0.5 (maximum impurity).
This node is very MIXED — lots of both survivors and non-survivors.
The algorithm will split here to reduce this impurity.

NUMBER OF TRAINING PASSENGERS in this node.
668 out of 668 → this is the ROOT node (all data arrives here).
After each split, child nodes have fewer samples.
Leaf nodes may have just 10-20 samples (min_samples_leaf).

CLASS DISTRIBUTION: [class 0 count, class 1 count]
411 = did not survive (class 0)
257 = survived (class 1)
Total: 411 + 257 = 668 ✓

MAJORITY CLASS — the prediction if we STOPPED here.
Since 411 > 257, the majority didn't survive.
All leaf nodes end in a class prediction.
For leaf nodes, this IS the final

```
Sex ≤ 0.5
gini = 0.4722
samples = 668
value = [411, 257]
class = 0
```

Sex ≤ 0.5

THE QUESTION being asked at this node.

gini = 0.4722

IMPURITY of this node BEFORE splitting.

samples = 668

NUMBER OF TRAINING PASSENGERS in this node.

value = [411, 257]

CLASS DISTRIBUTION: [class 0 count, class 1 count]

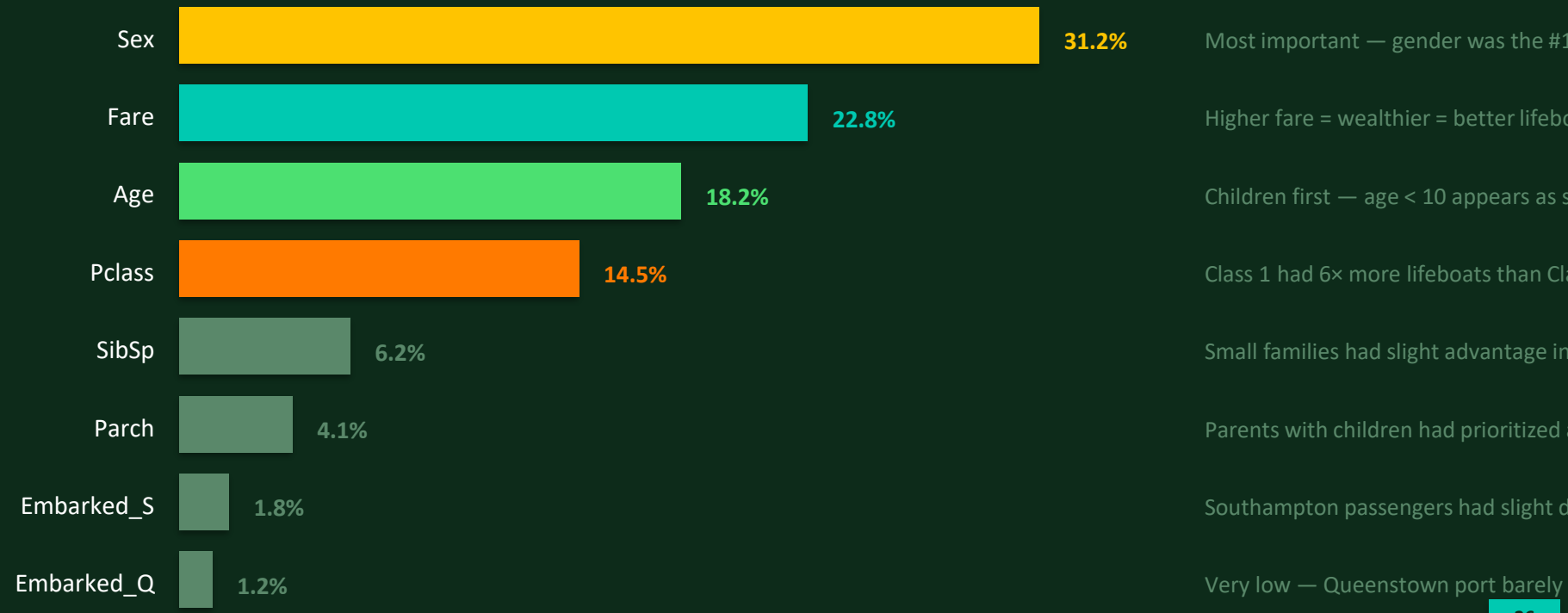
class = 0

MAJORITY CLASS — the prediction if we STOPPED here.

Feature Importance — What Did the Tree Learn?

Decision Trees rank features by how much they reduce impurity

```
feature_importances_  =  how much each feature reduced the weighted Gini across all splits
                                                (sum = 1.0)
```



10

Real-World Examples

Where Decision Trees are used in industry today

Decision Trees in the Real World

These algorithms make critical decisions in many industries every day

Banking & Credit

Credit scoring: 'If income < 30k AND debt > 50k → HIGH RISK'
Loan approval decisions that must be explained to applicants
Fraud detection rules that compliance officers can audit

Why DT? Regulators require explainable credit decisions by law

Legal & Compliance

'If transaction > $10,000 AND country in high_risk_list → report'
Insurance policy eligibility assessment
Tax fraud risk scoring with explainable criteria

Why DT? Legal requirements for explainable decisions (GDPR, Fair Credit)

Medical Diagnosis

'If age > 50 AND cholesterol > 240 AND smoker → screen for heart disease'
Decision support systems in emergency triage
Diagnostic flowcharts that doctors already use manually

Why DT? Doctors need to understand and trust the recommendations

Manufacturing

'If temperature > 85°C AND vibration > 0.8Hz → maintenance alert'
Quality control: automatically classify defective products
Sensor-based fault detection in production lines

Why DT? Engineers can read and validate the decision rules

Customer Churn

'If subscription > 3 months AND last_login > 30 days → at risk'
Priority call lists for customer retention teams
Automatic discount triggers based on behavior rules

Why DT? Business teams can act on the rules directly

Agriculture

'If soil_pH < 5.5 AND rainfall < 30mm → add fertilizer'
Crop disease diagnosis from leaf appearance
Optimal harvest timing based on weather conditions

Why DT? Farmers need simple, actionable, explainable advice

Decision Tree vs Other Models — Full Comparison

Understanding where DTs fit in the ML ecosystem

Property	Decision Tree	Random Forest	XGBoost	KNN	Log. Reg.
Accuracy (Titanic)	82.5%	81.6%	82.5%	78.9%	~80%
Training speed	Fast	Medium	Medium	Fast	Fast
Prediction speed	Very fast	Fast	Fast	Very fast	Slow
Interpretable?	✅ Yes	❌ No	❌ No	Partly	✅ Yes
Feature scaling?	❌ Not needed	❌ Not needed	❌ Not needed	✅ Required	✅ Required
Handles nonlinear?	✅ Yes	✅ Yes	✅ Yes	✅ Yes	❌ No
Overfits easily?	⚠ Yes	Less so	Less so	Yes (large K)	No
Missing values?	Some support	Some support	✅ Yes	❌ No	❌ No
Best for	Explainability	General use	Best accuracy	Simple data	Linear data

Complete Decision Tree Workflow — Step by Step

The professional process from raw data to trained model

1 Define the Problem

What are you predicting? (Survived/Not? Price? Category?)
Which metric matters? (F1? Precision? Recall?)
→ Titanic: Binary classification, use F1-Score

2 Prepare the Data

Clean missing values. Encode categorical features.
NO scaling needed for Decision Trees!
→ Sex: female=0, male=1. Embarked: one-hot encoding.

3 Split Train/Test

Use stratify=y to preserve class balance.
NEVER use test data for training!
→ train_test_split(X, y, test_size=0.25, stratify=y)

4 Start Simple

First: train with max_depth=3. Get a baseline score.
Then gradually increase depth and observe what changes.
→ This teaches you how sensitive the tree is.

5 Tune Hyperparameters

Try max_depth: 2, 3, 4, 5, 6. Watch train vs test gap.
Add min_samples_leaf to smooth the tree further.
→ Use GridSearchCV for systematic tuning.

6 Evaluate & Interpret

Read the classification_report. Check confusion matrix.
Visualize the tree — read the rules it learned!
→ Can you explain each prediction? You should be able to.

Common Mistakes — Avoid These!

Every beginner makes these — now you know better

✕ Not setting max_depth

DecisionTreeClassifier() with no max_depth = 100% training accuracy, 70% test.
Fix: Always start with max_depth=4. Tune from there.

✕ Ignoring class imbalance

63% didn't survive → tree biases toward 'not survived' to get easy accuracy.
Fix: Check class_weight='balanced' or use F1-Score instead of accuracy.

✕ Evaluating on training data

dt.score(X_train, y_train) = 99% — looks amazing! But this is LYING to yourself.
Fix: ALWAYS use dt.score(X_test, y_test) or classification_report(y_test, y_pred).

✕ Not checking both train and test

You MUST see both scores. train=85%, test=83% → good. train=99%, test=72% → problem!
Fix: Print both scores every time. Monitor the gap.

✕ Scaling features before DT

StandardScaler before a Decision Tree is pointless — trees use thresholds not distances.
Fix: Skip scaling for tree-based models. It wastes time and adds no value.

✕ Using one split only

One train/test split might get 'lucky' or 'unlucky' depending on who's in test set.
Fix: Use cross_val_score(dt, X, y, cv=5) for reliable performance estimate.

12

Summary & Cheat Sheet

Everything about Decision Trees on one page

All Formulas — Quick Reference

Everything you need to derive, build, and evaluate a Decision Tree

Gini Impurity

$Gini(node) = 1 - \Sigma(p_i)^2$

Range: 0 (pure) → 0.5 (max impurity)

Gini Cost (CART)

$Cost = (N_L/N) \times Gini_L + (N_R/N) \times Gini_R$

Pick the split with LOWEST cost

Entropy

$E(S) = -\Sigma\ p_i \times \log_2(p_i)$

Range: 0 (pure) → 1 (max impurity)

Information Gain

$IG(T,X) = E(T) - E(T,X)$

Pick the feature with HIGHEST gain

Accuracy

$Acc = (TP + TN) / Total$

Overall correct predictions

Precision

$Prec = TP / (TP + FP)$

Quality of positive predictions

Recall

$Recall = TP / (TP + FN)$

Coverage of actual positives

F1-Score

$F1 = 2 \times (P \times R) / (P + R)$

Harmonic mean: balanced metric

Feature Importance

$FI = \Sigma$ (weighted Gini reduction at node)

How much each feature reduced impurity

Proportion

$p_i = count(class\ i) / count(total)$

Used in Gini and Entropy formulas

Key Concepts Cheat Sheet

Every concept from this session — summarized

Root Node

Top node. The BEST first question. Only 1 per tree. Whole dataset arrives here.

Internal Node

Decision point inside the tree. Asks a question, splits into 2 branches.

Leaf Node

End point. No more splitting. This is the PREDICTION (class label).

Depth

Number of levels. max_depth=4 allows 4 questions max.

Gini Impurity

Measures how MIXED a node is. 0=pure, 0.5=maximum disorder.

Entropy

Alternative impurity measure. Uses $\log_2$. Range: 0 to 1.

Information Gain

Reduction in entropy after splitting. Pick highest gain feature.

CART

Classification And Regression Tree. sklearn's default DT algorithm. Uses Gini.

Overfitting

Tree memorizes training data. Train acc >> Test acc. Fix: max_depth.

Pruning

Reducing tree complexity to avoid overfitting. Pre-prune: set max_depth beforehand.

Feature Importance

Importance score for each feature. Sum = 1.0. Tracks Gini reduction.

Stratified Split

train_test_split(stratify=y) → same class % in both train and test sets.

THANK YOU

BrainerX Academy · Decision Trees — Complete Guide